

RoboAdmin — Product Requirements Document (PRD)

Product	RoboAdmin — AI-powered mortgage document operations platform
Document type	Full-stack PRD (frontend + backend)
Version	2.0
Status	Draft for engineering review
Owner	Product
Audience	Frontend eng, backend eng, ML eng, DevOps, design, QA
Proposed stack	FE: Next.js 15 · TS · Tailwind · shadcn/ui · Framer Motion · BE: Node/TypeScript (NestJS or Fastify) · PostgreSQL · Redis · object storage (S3-compatible) · message queue · AI: OCR + classification service

Technology picks marked *proposed* are recommendations, not mandates — confirm with the team (see §20). Behaviour and contracts in this doc are the requirement; implementation may vary.

1. Summary

RoboAdmin turns the chaos of mortgage paperwork into an observable, AI-driven assembly line. Documents arrive (mostly as email attachments), and a backend pipeline **captures, analyses, classifies, renames, routes, and queues them for human review** before filing them into the firm's connected storage. The frontend is the **control room** where firms supervise this; the backend is the **engine** that ingests email, runs AI, enforces tenant isolation, and writes to storage providers.

This PRD covers the **full product across frontend and backend**. Phase 1 ships a thin but real end-to-end slice (see §18); later phases harden scale, connectors, and ML quality.

2. Problem & opportunity

Mortgage brokers receive high volumes of client documents (passports, payslips, bank statements, proof of address) via email and portals. Today a human opens each attachment, identifies it, renames it to the firm's convention, files it in the right client/case folder in OneDrive/Drive/etc., and tracks what's missing. It's slow, error-prone, and a compliance risk.

Opportunity: automate 90%+ of this with AI, keep a human in the loop only where confidence is low, give the firm a live auditable view, and file securely into the tools they already use — all with strict per-firm isolation.

3. Goals, non-goals, success metrics

Goals

- Real end-to-end pipeline: email capture → AI processing → review → filing into connected storage.
- Strict **multi-tenant isolation** enforced at the data layer (not just UI).
- Best-in-class **AI Review Queue** with human-in-the-loop feedback that improves the model.
- Auditable, compliant handling of sensitive PII.

Non-goals

- Building proprietary foundation models (use OCR + hosted/fine-tuned classifiers). No native mobile apps. No CRM/lead-gen features.

Success metrics

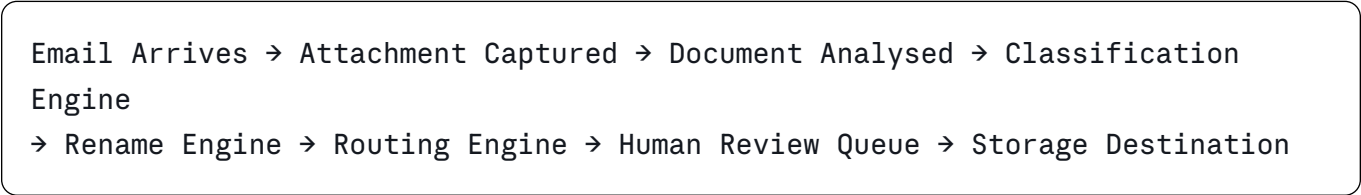
Metric	Target	Owner
Documents auto-classified without human edit	≥ 85%	ML
Classification accuracy (post-review)	≥ 95%	ML
Median capture → reviewed	< 2 min	BE
% requiring manual review	< 20%	ML
Pipeline stage success (no DLQ)	≥ 99.5%	BE
API p95 latency (reads)	< 300 ms	BE
Cross-tenant data leakage incidents	0	BE/Security

4. Personas

Persona	Role	Goals	Pain today
Priya — Practice Principal	Firm owner / Admin	Oversight, capacity, audit-readiness	No visibility; compliance anxiety
Daniel — Mortgage Adviser	Works cases	Know what each client is missing	Buried in email; forgets outstanding docs
Maria — Operations / Document Admin	Power user	Clear high-volume queue fast & accurately	Manual rename/file; easy to misfile
Tom — Compliance	Audit & oversight	Prove correct, isolated handling	No reliable trail
System — the pipeline	Automated actor	Process documents reliably at scale	n/a

5. Core concept, pipeline & glossary

The **document processing pipeline** is the heart of the product, implemented as an **event-driven sequence of backend stages**, surfaced live in the UI:



Six document **states** shown in the UI / persisted in the DB: Captured → Analysed → Classified → Renamed → Routed → Reviewed (then Filed).

Term	Meaning
Firm (tenant)	Isolated brokerage. <code>tenant_id</code> on every record; no cross-firm access.
Workspace	Scope within a firm (branch/team).
Case	A mortgage application tied to a client, moving through mortgage stages.
Document state	Pipeline position; drives UI + worker routing.
Confidence score	AI certainty 0-100 per stage; gates auto-approval vs review.
Connector	An integration: email source (Outlook/Gmail/IMAP) or storage destination (OneDrive/Drive/Dropbox/Box/Built-In).
Storage destination	Where the final file is written.

6. User scenarios (FE + BE annotated)

A — Happy path (high confidence)

Client emails `scan001.pdf`.

1. **BE (Ingestion):** Email connector receives a Graph/Gmail webhook → downloads attachment → virus scan → persists raw file to object storage → emits `document.captured`. **FE:** appears in Activity Feed as `Captured`.
2. **BE (Pipeline workers):** `Analyse` (OCR + field extraction) → `Classify` ("UK Passport", 97%) → `Rename` (`Smith_John_Passport_2026-06.pdf`) → `Route` (folder `Clients/John Smith/ID`, destination OneDrive). Each stage emits an event and updates state + confidence.
3. **BE:** confidence ≥ firm auto-approve threshold → item enters Review Queue flagged green (or auto-approves per rule). **FE:** queue shows green confidence ring.
4. **Maria** presses `a` → **BE:** writes file to OneDrive via Storage connector → emits `document.filed` → case "ID" requirement satisfied. **FE:** completion ring updates live.

B — Low confidence (hero flow)

`IMG_2231.jpg`, classified "Bank Statement?" at 64%.

- **BE:** confidence < threshold → `review_required`; surfaces in queue with amber flag. **FE:** sorts to top, glow, shows original→suggested diff.
- **Maria** corrects to "Proof of Address", edits folder, approves. **BE:** persists correction, files document, **logs the human correction as a labelled training example** and writes an immutable audit entry.

C — Missing documents

Daniel opens John Smith's case. **BE:** case requirements engine computes which document types are expected vs received and returns completion %. **FE:** completion ring 80% + amber "payslip missing" flag + stage stepper at Valuation. He messages the client (communication logged).

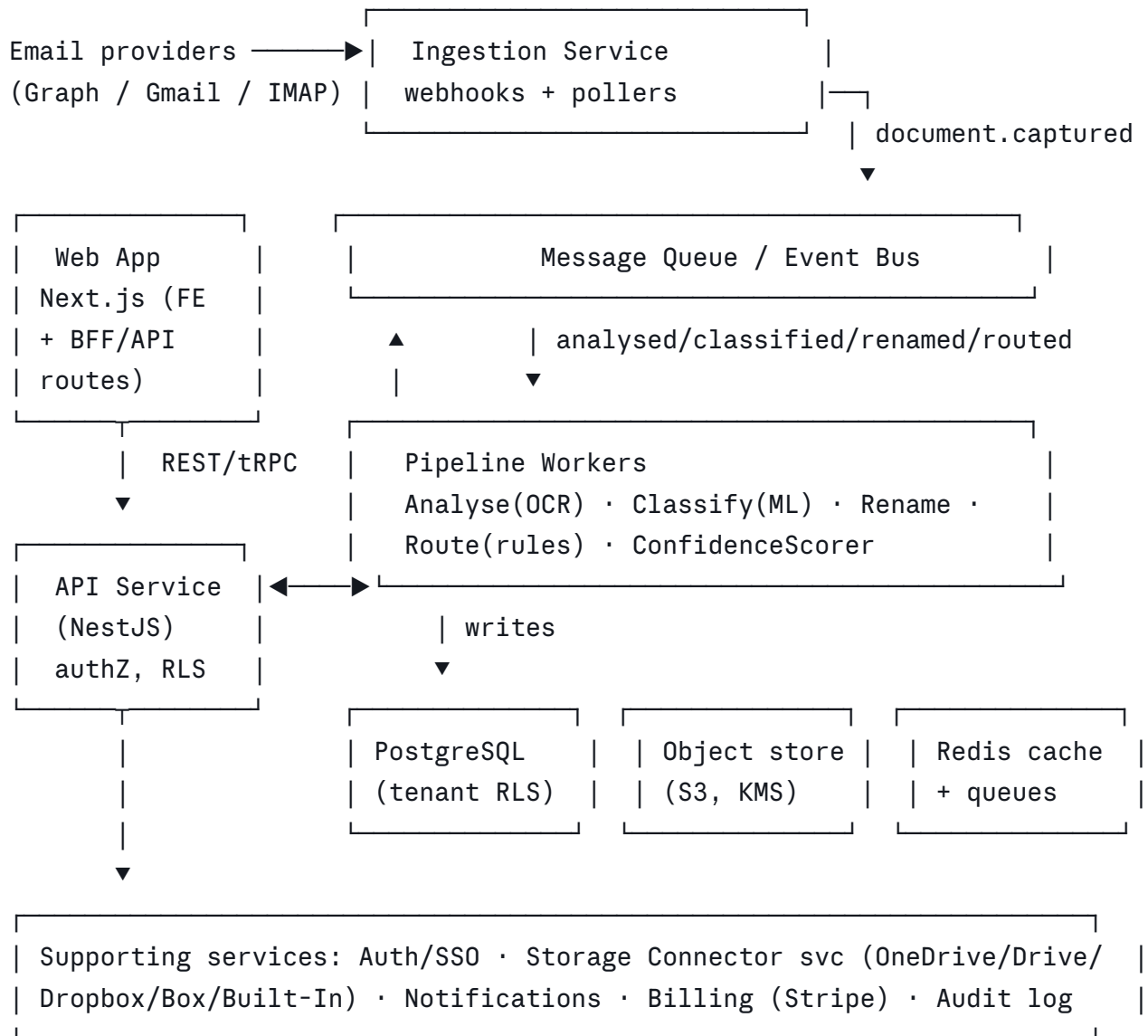
D — Onboarding a firm

Priya connects Outlook (**BE:** OAuth flow, store encrypted tokens, register webhook/poller, set status `Monitoring`) and OneDrive (**BE:** OAuth, verify write scope, sync health check). Sets classification + naming rules (**BE:** persisted per tenant; consumed by Rename/Route engines). **FE:** context strip reflects the firm everywhere.

E — Bulk backlog

42 documents awaiting review. **FE:** filter ≥90%, select all, "Approve all". **BE:** batch endpoint processes approvals idempotently, files each to its destination, returns per-item results; partial failures surface without blocking the batch.

7. System architecture (proposed)



Can ship as a **modular monolith** (single deployable, internal modules + worker process) and split into services later. The event bus + worker model is the non-negotiable part — the pipeline must be asynchronous, retryable, and idempotent.

8. Data model (core entities)

Every tenant-scoped table carries `(tenant_id)` (firm) and is protected by **Postgres Row-Level Security**.

Entity	Key fields
Firm (tenant)	id, name, branding, plan_id, settings, data_region
Workspace	id, tenant_id, name
User	id, tenant_id, email, role, status, sso_subject
Client	id, tenant_id, name, contact, personal_details
Case	id, tenant_id, client_id, adviser_id, mortgage_stage, status, completion_pct
CaseRequirement	id, case_id, document_type, status (required/received/waived)
Document	id, tenant_id, case_id?, client_id?, original_filename, current_filename, mime, size, state, storage_key, source_connector_id
DocumentClassification	id, document_id, suggested_type, confidence, model_version, final_type, corrected_by
DocumentRouting	id, document_id, suggested_folder, destination_connector_id, final_path
ReviewItem	id, document_id, status (pending/approved/rejected), confidence, reviewer_id, reason, decided_at
Connector	id, tenant_id, kind (email/storage), provider, status, sync_health, last_sync_at, encrypted_credentials
ClassificationRule	id, tenant_id, match (sender/filename/keyword), set_type, route_to, naming_template
AuditEvent	id, tenant_id, actor, action, entity, before/after, ip, ts (append-only)
Notification	id, tenant_id, user_id, type, payload, read_at
Subscription / UsageMeter	id, tenant_id, plan, seats, docs_processed, period

9. Processing pipeline — backend specification

Each stage is a worker consuming an event and emitting the next. **All processing is idempotent** (keyed by `document_id` + stage) and **at-least-once** with dedup.

Stage	Input	Work	Output / event	Confidence
Capture	email webhook / upload	dedup (hash), virus scan, persist raw to object store	<code>document.captured</code> , state=Captured	n/a
Analyse	captured doc	OCR + text/field extraction, detect page count, language	<code>document.analysed</code> , extracted metadata	extraction quality
Classify	analysed doc	ML/LLM classifier → document type	<code>document.classified</code> , suggested_type	0-100
Rename	classified doc	apply firm naming template using extracted fields	<code>document.renamed</code> , suggested_filename	template-fill completeness
Route	renamed doc	rules engine → target folder + storage destination	<code>document.routed</code> , suggested_folder	rule match strength
Confidence gate	routed doc	min(stage confidences) vs firm threshold	auto-approve OR <code>document.review_required</code>	composite
Review	review item	human approve/ reject/edit	<code>document.reviewed</code> (+ training label)	n/a
File	approved doc	write to storage connector, verify, record path	<code>document.filed</code> , state=Filed	n/a

Reliability requirements: retries with backoff per stage; failed messages → **dead-letter queue** with alert; **idempotency keys** prevent duplicate capture/filing; a document's full state history is queryable for the timeline UI; processing must survive worker restarts (no in-memory-only state).

10. Integrations

10.1 Email connectors (ingestion)

Provider	Mechanism	Notes
Outlook / Microsoft 365	Microsoft Graph subscriptions (webhooks) + delta polling fallback	OAuth2; <code>Mail.Read</code> ; renew subscriptions
Gmail / Google Workspace	Gmail API push (Pub/Sub) + history polling	OAuth2; restricted scopes; verification
IMAP	IDLE / scheduled polling	App passwords; manual host/port config

Requirements: capture only attachments (configurable types/size); store source message metadata; per-connector **monitoring status** + **attachment capture stats**; encrypted token storage; token refresh; graceful degradation to polling; surface auth errors to the connector card.

10.2 Storage connectors (destinations)

Provider	API	Capability
OneDrive / SharePoint	Graph	create folders, upload, check usage
Google Drive	Drive API	same
Dropbox	Dropbox API	same
Box	Box API	same
Built-In	internal object store	default for trials

Requirements: OAuth + scope verification; folder creation honoring routed path; **sync health** + **last sync** + **storage usage** reporting; write verification (confirm file exists post-upload); conflict handling (dedupe/version); retry on transient failures.

11. AI / ML requirements

- **OCR / extraction:** hosted OCR (e.g. document AI) or self-hosted; extract raw text + structured fields (names, dates, account numbers) used by Rename.
 - **Classification:** start with an LLM/zero-shot or off-the-shelf classifier over a fixed taxonomy of mortgage document types; emit type + calibrated confidence + `model_version`.
 - **Confidence calibration:** thresholds configurable per firm (default review if < 90 ; band colours 90 / 70 — confirm in §20).
 - **Human-in-the-loop:** every correction in the Review Queue is stored as a labelled example; a feedback dataset enables periodic re-training/eval. Track accuracy by document type.
 - **PII handling:** documents contain sensitive PII — never log document contents; redact in traces; restrict model providers to compliant/region-appropriate options; support a no-retention inference mode.
 - **Versioning & eval:** model versions are recorded per classification; an eval harness reports precision/recall per type before promoting a model.
-

12. API design (representative)

REST/JSON (or tRPC); all endpoints tenant-scoped via auth context; pagination, filtering, idempotency keys on writes.

```

# Cases & clients
GET    /api/cases?stage=&adviser=&missingDocs=true
GET    /api/cases/:id                # case + timeline + requirements +
completion
GET    /api/clients/:id

# Documents
POST   /api/documents                # manual upload → enters pipeline
(multipart)
GET    /api/documents?state=&type=&caseId=
GET    /api/documents/:id           # detail + state history + classification +
routing

# Review queue (hero)
GET    /api/review?confidenceMax=&type=
POST   /api/review/:id/approve      # body: optional edits (type, filename,
folder)
POST   /api/review/:id/reject       # body: reason
POST   /api/review/bulk             # body: ids[] + action → per-item results

# Connectors
POST   /api/connectors/email/:provider/oauth/start
GET    /api/connectors/email/:provider/oauth/callback
POST   /api/connectors/storage/:provider/oauth/start
GET    /api/connectors              # status, health, last sync, usage

# Automation, settings, metrics, billing
GET/PUT /api/automation/rules
GET/PUT /api/settings/*             # firm, users, roles, classification
rules, branding
GET    /api/metrics/dashboard       # all dashboard tiles + pipeline counts
GET    /api/billing/subscription, POST /api/billing/checkout

# Inbound webhooks (provider → us)
POST   /api/webhooks/email/:provider
POST   /api/webhooks/storage/:provider
POST   /api/webhooks/billing        # Stripe

```

Domain events (internal bus): document.captured | analysed | classified | renamed |
routed | review_required | reviewed | filed | failed. **Outbound to FE:** live updates for
Activity Feed / queue counts via SSE or WebSocket.

13. Auth, roles & multi-tenancy

- **AuthN:** email/password + SSO (Google/Microsoft); proposed via a provider (WorkOS/Clerk/Auth0) for SSO + SCIM later.
- **Tenant resolution:** every request resolves a `tenant_id` from the session; **all queries filtered by it**; **Postgres RLS as defence-in-depth**; object storage keys namespaced per firm; connector tokens encrypted per tenant (KMS).
- **RBAC matrix (proposed):**

Capability	Owner/ Admin	Adviser	Ops/Doc Admin	Compliance (RO)
View dashboard/cases/clients	✓	✓	✓	✓
Approve/reject in Review Queue	✓	✓	✓	X
Manage connectors	✓	X	✓	X
Edit classification/automation rules	✓	X	✓	X
Manage users/roles/billing	✓	X	X	X
View audit log	✓	X	X	✓

14. Functional requirements by module (FE + BE)

Module	Frontend responsibilities	Backend responsibilities
Dashboard	Metric tiles, animated pipeline, activity feed, AI insights; live updates	<code>/metrics/dashboard</code> aggregations; event stream (SSE/WS); compute throughput & accuracy
AI Review Queue (hero)	Confidence sorting/colour, original→suggested diff, keyboard ops, bulk bar, empty state	Queue query, approve/reject/bulk endpoints (idempotent), persist corrections + training labels + audit, trigger filing
Document Centre	Folder tree, search, filters, drag-drop upload, state + destination badges	Upload→pipeline, search/filter API, state history, signed download URLs
Case Management	Case list, workspace, timeline, stage stepper, completion ring, missing-doc flags	Requirements engine (expected vs received), completion %, timeline assembly, adviser assignment
Client Management	Profile, associated cases/docs, comms timeline, progress	Client CRUD, comms log, aggregation
Storage Connectors	Connector cards, status/health/usage, connect/reconfigure	OAuth flows, token storage, sync health checks, usage polling, write verification
Email Connectors	Cards, monitoring status, capture stats, IMAP config sheet	OAuth/IMAP setup, webhook/poller management, attachment capture, stats
Automation Centre	Visual flow builder, node config, toggles, run history	Persist rules, evaluate rules in pipeline, run history events
Billing	Plan comparison, usage meters, invoices, upgrade CTA	Stripe integration, usage metering, plan-limit enforcement, webhooks
Settings	Tabs: firm, users, roles, storage, email, rules, branding, notifications	CRUD + validation per tenant; branding feeds FE theming; rules feed pipeline

15. Non-functional requirements

Security & compliance

- Encryption in transit (TLS) and at rest (KMS); object storage encrypted; connector credentials encrypted per tenant.
- **GDPR/UK:** lawful basis, data subject access & erasure, configurable **retention** and deletion; **data residency** per firm region; DPA-friendly subprocessors.
- Documents are sensitive PII: never logged; access audited; least-privilege; **virus scanning** on capture; signed, expiring download URLs.
- **Immutable audit log** for every state change and human action.
- Rate limiting, input validation, OWASP top-10 hardening, secrets manager, dependency scanning.

Performance & scale

- API reads p95 < 300 ms; dashboard aggregates cached.
- Pipeline horizontally scalable workers; target throughput defined per plan; backpressure via queue.
- Lists of 500+ items virtualised on FE; server pagination everywhere.

Reliability

- $\geq 99.5\%$ stage success; DLQ + alerting for failures; retries with backoff; idempotent capture & filing; graceful connector degradation.
- Target availability 99.9% for the API; documents never lost (durable object store + state history).

Observability

- Structured logs (PII-safe), metrics (throughput, stage latency, confidence distribution, queue depth, DLQ size), distributed tracing, alerting on SLA breaches.

Accessibility & responsive (FE): WCAG 2.1 AA on dark theme; full keyboard ops (Review Queue); `prefers-reduced-motion`; desktop/tablet/mobile behaviours.

16. States & edge cases

- **FE states:** empty (geometric cyan glyph + action), loading (layout skeletons), error (red panel + retry + mono code), processing (scan line/indeterminate bar).
 - **System edge cases:** duplicate attachment (hash dedup); unsupported/oversized file (reject + notify); corrupt/unreadable scan (low extraction confidence $\rightarrow$ review); connector token expired mid-sync (degrade + surface error); storage write fails (retry $\rightarrow$ DLQ $\rightarrow$ flag); 0%/100% confidence; very long filenames (truncate + tooltip); firm with no connectors (guided empty state); deleted/merged case re-routing.
-

17. Billing & metering (backend)

Stripe-backed. Plans **Free** / **Starter** / **Professional** / **Business** differ by monthly document volume, seats, connector count, and features. Backend **meters usage** (documents processed, seats, connectors) and **enforces plan limits** (soft warn → hard block with upgrade prompt). Stripe webhooks update subscription state; invoices surfaced read-only in UI.

18. Phasing / out of scope

Phase	FE	BE
P1 — Thin slice	Shell, Dashboard, Review Queue, Document Centre (mock + real where ready)	Auth + tenancy + RLS, capture via 1 email provider, full pipeline (basic ML), Built-In storage, review/approve→file
P2 — Connectors	Connector + Automation UIs	Remaining email + storage providers, rules engine, sync health
P3 — Cases/ Clients/ Compliance	Case workspace, client profiles, audit views	Requirements engine, comms, immutable audit, retention
P4 — Billing & scale	Billing, settings depth	Stripe metering, autoscaling workers, ML re-training loop

Out of scope (all phases): native mobile apps, CRM/lead-gen, building foundation models, e-signature.

19. Milestones (parallel FE/BE tracks)

Milestone	FE	BE	Done when
M1 Foundation	Tokens, shell, Command-K, tenant strip	Auth, tenancy, RLS, DB schema, event bus skeleton	Login → scoped empty app
M2 Pipeline core	Pipeline + state components, Activity feed	Capture (1 provider) → all stages → Built-In filing	Scenario A end-to-end
M3 Review Queue	Confidence UI, diffs, keyboard, bulk	Queue + approve/reject/bulk + training labels + audit	Scenarios B & E pass
M4 Documents/ Cases/Clients	Centre, case workspace, client profiles	Search, requirements engine, completion, comms	Scenario C passes
M5 Connectors/ Automation	Connector + flow-builder UIs	All providers, rules engine, sync health	Scenario D passes
M6 Billing/ Settings/hardening	Billing, settings, states pass	Stripe, metering, observability, DLQ, security review	Production-ready

20. Dependencies & open questions

- **Auto-approval:** may documents auto-file above a confidence threshold, or is human approval always required? (Changes gate logic + compliance posture.)
- **Confidence thresholds:** confirm review cut-off and band colours (assumed 90/70).
- **ML approach:** off-the-shelf classifier vs fine-tuned vs LLM zero-shot for v1? OCR provider choice?
- **Data residency:** single region (UK) or per-firm? Affects infra + subprocessor selection.
- **Stack confirmation:** NestJS vs Fastify; queue choice (SQS/Kafka/Redis Streams); auth provider; hosting.
- **Mortgage stage list** for the stepper (assumed Application → AIP → Valuation → Offer → Completion).
- **Plan limits:** exact volume/seat/connector caps per tier.
- **Compliance:** required certifications (e.g. SOC 2, Cyber Essentials) and timeline.

21. Definition of Done

Login with enforced tenant isolation (verified: zero cross-firm access); a real document flows email → capture → analyse → classify → rename → route → review → filed into a connected store; the 5 scenarios in §6 are demonstrable; confidence gating, bulk review, audit logging, and plan-limit enforcement work; FE empty/loading/error states and responsive behaviour complete; security review and ML eval thresholds passed; observability dashboards live with alerting.